



Async calls with JavaScript

making a promise

What?

- Sometimes JavaScript needs some time
 - retrieving data from a database
 - reading a file
 - submitting a form
- You can't know exactly how long this action will take
- But you want to give the user some feedback (no frozen browser)

SOLUTION: Use async calls



Sync

Sync

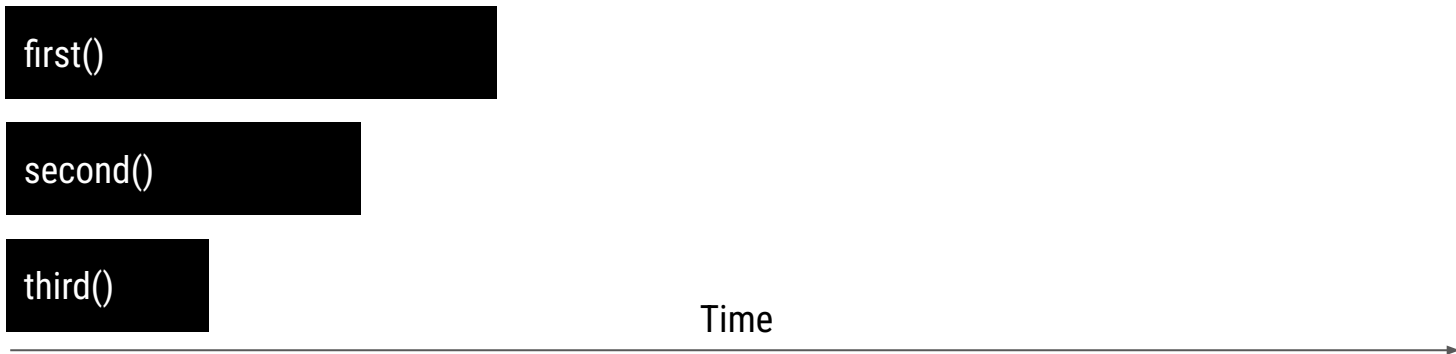
- Calling a function → is added to the “call stack”
- The next function is called when the previous function completes



Async

Async

- Calling a function → is added to the “call stack”
- Possibility to execute a next function, while the previous one is executing
- Execution of the original function continues



Async

- `setInterval()`, `setTimeout()` are async functions
- `first()`, `second()` and `third()` are called directly after each other
- 2 processes are executing simultaneously
 - sequence of `first()`, `second()` and `third()`
 - `setInterval()` scheduling / callback



Async: problem

- `decodeMessage()` needs 3 seconds to process
NOTE: this is a simulation of “heavy processing” using `setTimeout()`
- But the return function does not wait for `decodeMessage()` to finish
- Resulting in an empty string
- Have a close look at the order of the logs:

■ Start the application!	script.js:22
■ Start decoding, will take a while...	script.js:11
■ The message is:	script.js:25
■ The application finished executing	script.js:26
■ Finished decoding	script.js:15
■ Result: This is a secret message!	script.js:16



SOLUTION: using “promises”

Demo 03-async-problem

Promises

Promises

- When a result isn't available immediately
 - Retrieving data from a server, using `setTimeout`, ...
- While we wait, the function returns a “promise”
- This promise has one of the following states:
 - **pending**: no result yet, promise hasn't been completed
 - **fulfilled**: there's a result, promise has completed
 - **rejected**: promise was executed, but there was an error



Promises

- `decodeMessage()` does not return a string, but a Promise instead
- Within this Promise, you're "returning" by using the `resolve()` callback
 - You may also use `reject()` if you want to "return" an error
- This Promise is pending at first, and resolves after 3 seconds

■ Start the application!	script.js:25
■ Start decoding, will take a while...	script.js:14
■ The message is: [object Promise]	script.js:28
▶ <code>Promise {<pending>}</code>	script.js:29
■ The application finished executing	script.js:30
■ Finished decoding	script.js:17
■ Result: This is a secret message!	script.js:18

■ Start the application!	script.js:25
■ Start decoding, will take a while...	script.js:14
■ The message is: [object Promise]	script.js:28
▼ <code>Promise {<pending>}</code> ⓘ	script.js:29
▶ <code>[[Prototype]]: Promise</code>	
<code>[[PromiseState]]: "fulfilled"</code>	
<code>[[PromiseResult]]: "This is a secret message!"</code>	
■ The application finished executing	script.js:30
■ Finished decoding	script.js:17
■ Result: This is a secret message!	script.js:18



Promises: problem

- The order of the messages is still wrong
 - “The application finished executing...” logs before “Finished decoding...”
- You need to handle the Promise correctly

SOLUTION: use `async` / `await` keywords



async / await

async / await

- The function, wanting to wait for a promise, needs the **async** keyword
- When you want to wait for the promise to resolve, add the **await** keyword
- Result:
 - Our program waits at the await keyword, until decodeMessage() finishes
 - When the Promise resolves, the program continues
 - NOTE: our init() function progresses as usual, as there is no async / await there!

```
const init = async () => {  
  console.log(`■ Start the application!`);  
  const secret = `!egassem terces a si sihT`;   
  const result = await decodeMessage(secret);  
  console.log(`■ The message is: ${result}`);  
  console.log(`■ The application finished executing`);  
};
```

■ Start the application!	script.js:22
■ Start decoding, will take a while...	script.js:11
■ Finished decoding	script.js:14
■ Result: This is a secret message!	script.js:15
■ The message is: This is a secret message!	script.js:25
■ The application finished executing	script.js:26



What do you need to remember?

- No panic: you don't need to write Promises yourself
- You only need to be able to use the **async** and **await** keywords
- If you are using 'await' in a function, that function needs to be 'async'
 - `const init = async () => { ... }`



async / await: the 2 rules

// Rule 1: add async to the function

```
const loadData = async () => {
```

// Rule 2: add await before a Promise

```
  const result = await someAsyncFunction();
```

```
  console.log(result);
```

```
};
```

// forgetting async -> SyntaxError

// forgetting await -> you get a Promise object not a value



fetch

fetch

- A JavaScript API to load external resources (eg: text, JSON...)
- Works over HTTP(S)
- Returns a Promise
- See [MDN](#)
- A two-step process:
 - **Step 1:** wait for the Response object
 - **Step 2:** retrieve the contents of the Response object

```
const getSongtext = async () => {  
  console.log(`Started loading the txt file`);  
  const response = await fetch(`assets/data/songtext.txt`);  
  console.log(response);  
  const lyrics = await response.text();  
  console.log(lyrics);  
  console.log(`Finished processing the txt file`);  
}
```



Demo 06-example-fetch

fetch: response methods

- `text()` → textual data
- `json()` → json encoded data (javascript objects as text files!)
- `blob()` → binary data (images, video, ...)
- `arrayBuffer()` → data as an array of bytes
- `formData()` → key/value pairs of a form



json

JSON

- A **text** based format
- **platform / language independent**
 - usable on Windows, Linux, Mac
 - usable in JavaScript, PHP, C#, Python,...
- **2 principles:**
 - Collection of **key-value pairs** (object)
 - **List** with values (array)

All info at <https://www.json.org/>



JSON

JSON ≠ JavaScript Object

- **JSON:** Property name (the key) needs to be wrapped in quotes “ ”
- **JSON:** Types are limited to String, Number, Array, true, false, null or another (JSON) object
 - eg: a Date object is displayed as a String
- **JSON:** Values cannot be functions



fetch: JSON

- reading a JSON file / service
- uses the `json()` method
- you can use the resolved item as a regular JavaScript object!

```
const getData = async () => {  
  console.log(`Start loading the JSON file`);  
  const response = await fetch(`assets/data/nero.json`);  
  const data = await response.json();  
  console.log(`Finished loading the JSON`);  
  
  document.querySelector(`.artist`).textContent = data.artist;  
  document.querySelector(`.album`).textContent = data.album;  
  document.querySelector(`.lyrics`).innerHTML = data.lyrics.map(line => `

${line}</p>`).join('');  
};


```



JSON in practice

Example: <http://omdbapi.com>

Example: <https://vaernewijck.github.io/json-statham/>



Demo 08-example-fetch-url-json

DevTools 🔥 🔥 🔥

Network tab

